

Regression/Classification with Linear Models

Machine Learning

Nomenclature for supervised learning

$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$ features (aka: inputs, independent variables)

y output (aka: response, dependent variable)

$y \in \mathbb{N}$ classification

$y \in \mathbb{R}$ regression

$h_{\theta}(x)$ hypothesis (prediction)

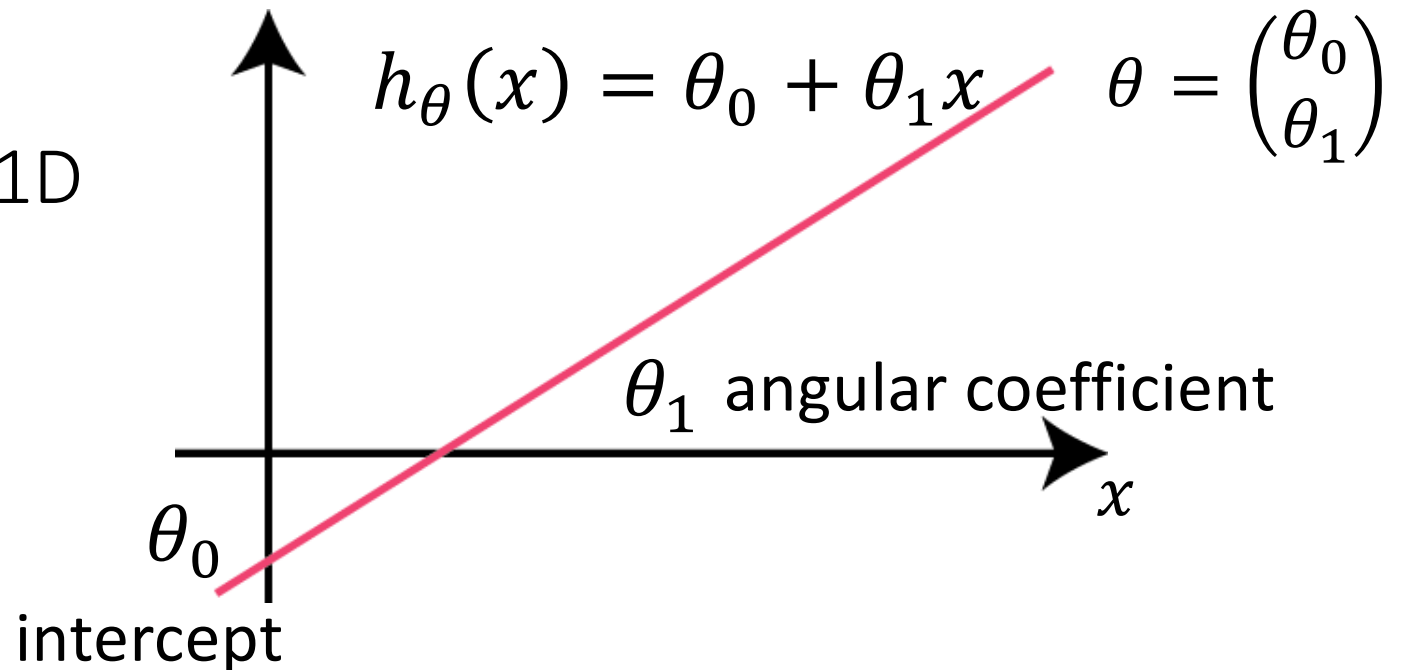


parameters of the algorithm

Linear Regression: the hypothesis is a linear function of the features

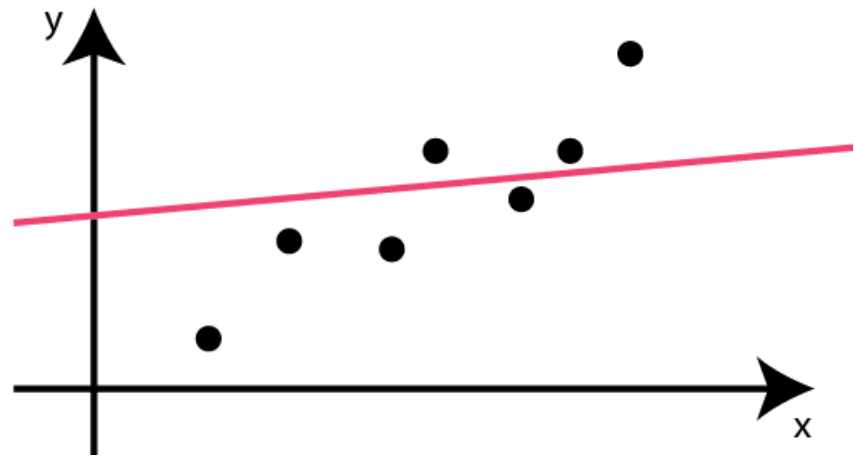
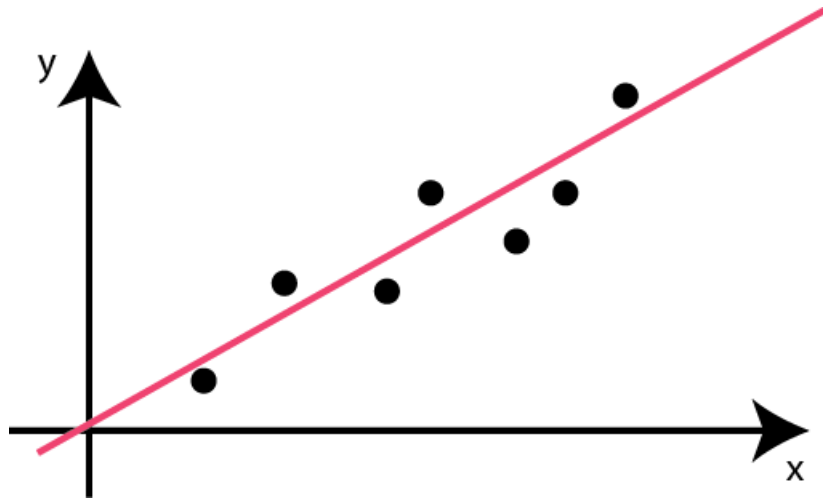
Linear Regression in 1D

$$x \in \mathbb{R}$$

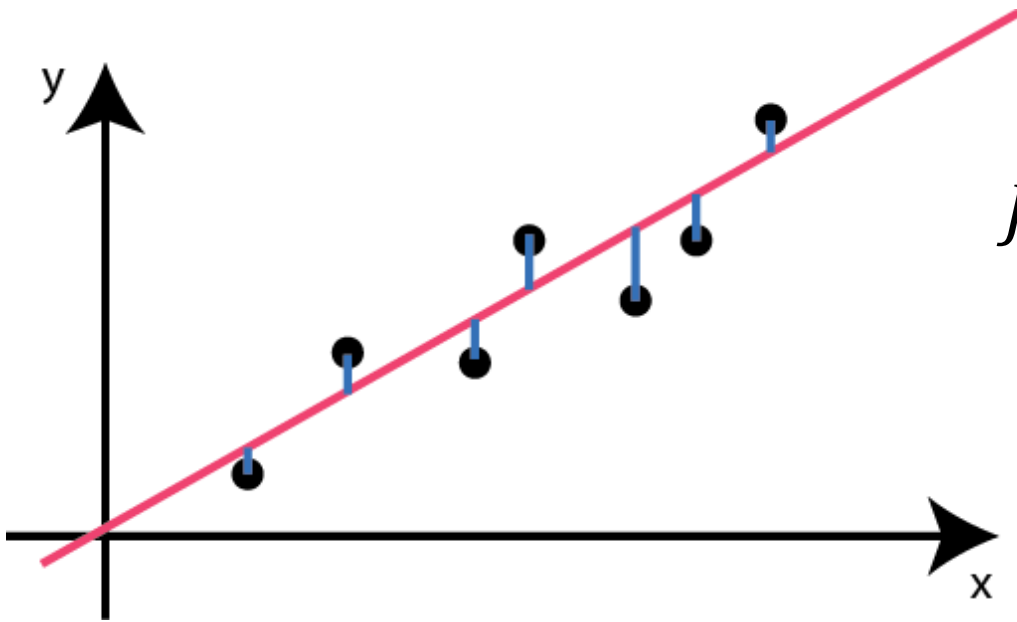


$h_{\theta}(x) = \theta_0 + \theta_1 x$ How do we choose the parameters ?

Minimize the difference between the hypothesis and the output in the training set



$h_{\theta}(x) = \theta_0 + \theta_1 x$ How do we choose the parameters ?



$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^{m-1} (h_{\theta}(x^i) - y^i)^2$$

COST FUNCTION

Linear Regression with multiple features

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

$$h_{\theta}(x) = \theta_0 1 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$
$$x_0 = 1$$

Linear Regression – Vectorial form

$$\Theta = \begin{pmatrix} \theta_0 \\ \vdots \\ \theta_n \end{pmatrix} \quad x = \begin{pmatrix} x_0 \\ \vdots \\ x_n \end{pmatrix} \quad h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i = \Theta^T x$$

$$X \overset{\text{samples}}{\parallel} \begin{pmatrix} x_0^0 & \dots & x_n^0 \\ \vdots & \ddots & \vdots \\ x_0^{m-1} & \dots & x_n^{m-1} \end{pmatrix} = \begin{pmatrix} (x^1)^T \\ \vdots \\ (x^m)^T \end{pmatrix} \quad X\Theta = \begin{pmatrix} h_{\theta}(x^0) \\ \vdots \\ h_{\theta}(x^{m-1}) \end{pmatrix} \quad Y = \begin{pmatrix} y^0 \\ \vdots \\ y^{m-1} \end{pmatrix}$$

$\xleftarrow{\text{features}}$

$$J(\Theta) = \frac{1}{2m} \sum_{i=0}^{m-1} (h_{\theta}(x^i) - y^i)^2$$

$$J(\Theta) = \frac{1}{2m} (X\Theta - Y)^T (X\Theta - Y)$$

Gradient of the cost function – Vectorial form

$$\begin{aligned}\frac{\partial J}{\partial \theta_j} &= \frac{1}{2m} \sum_i \frac{\partial (h_\theta(x^i) - y^i)^2}{\partial \theta_j} = \frac{1}{m} \sum_i (h_\theta(x^i) - y^i) \frac{\partial h_\theta(x^i)}{\partial \theta_j} \\ &= \frac{1}{m} \sum_i (h_\theta(x^i) - y^i) x_{ij}\end{aligned}$$

$$\nabla_{\theta} J = \frac{1}{m} X^T (X\Theta - Y)$$

$$\Theta' = \Theta - \frac{\alpha}{m} X^T (X\Theta - Y)$$

Gradient Descent

Normal Equation

$$\nabla_{\theta} J = 0$$

$$X^T (X\Theta - Y) = 0$$

$$X^T X \Theta = X^T Y$$

$$\Theta = (X^T X)^{-1} X^T Y$$

- It's an exact solution
- It could be impossible to invert the matrix

Logistic Regression - Hypothesis

$$y \in \{0,1\}$$

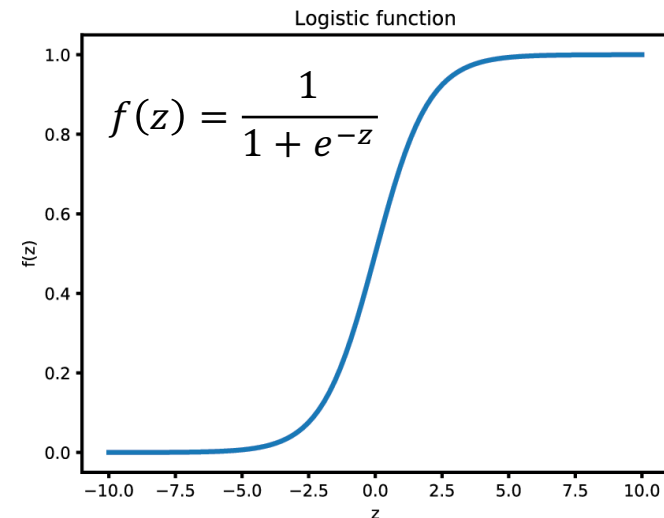
$$h_{\theta}(x) = P(y = 1|x; \theta)$$

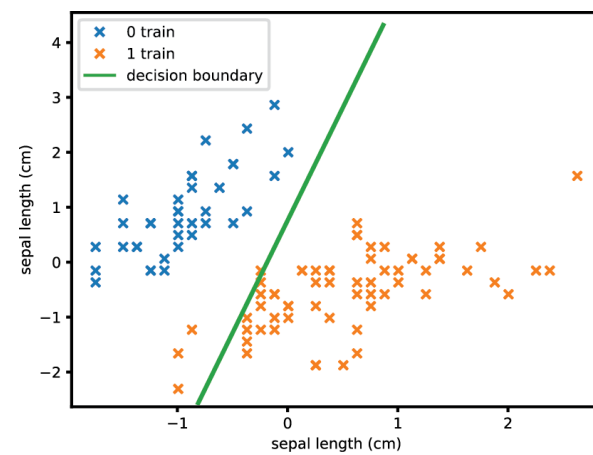
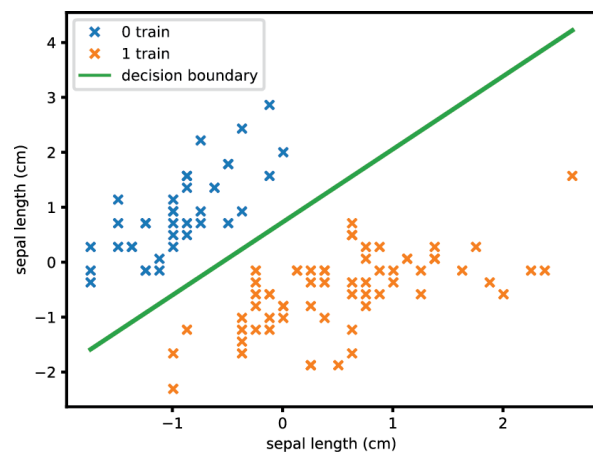
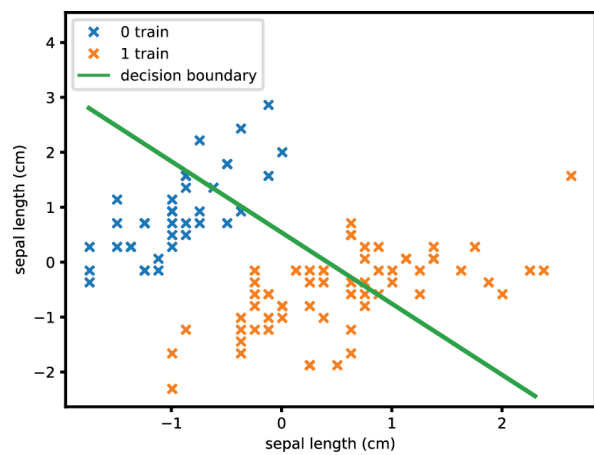
Probability that the sample with value x belongs to class 1, when using the parameters θ

$$h_{\theta}(x) = f(\Theta^T x) = \frac{1}{1 + e^{-\Theta^T x}}$$

$$\Theta^T x = 0 \rightarrow f(\Theta^T x) = 0.5$$

Decision boundary





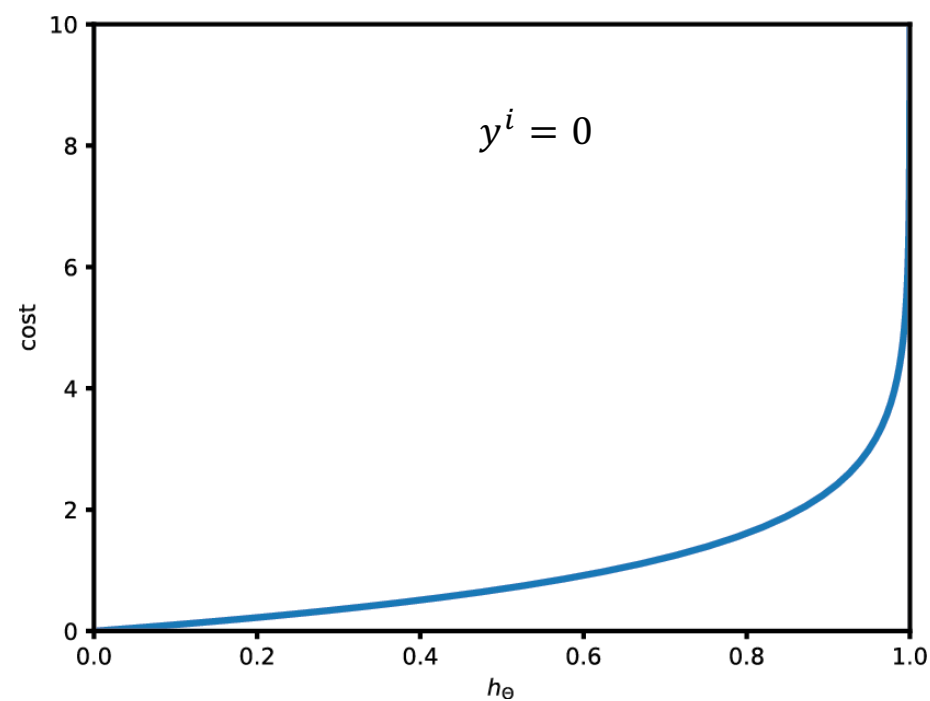
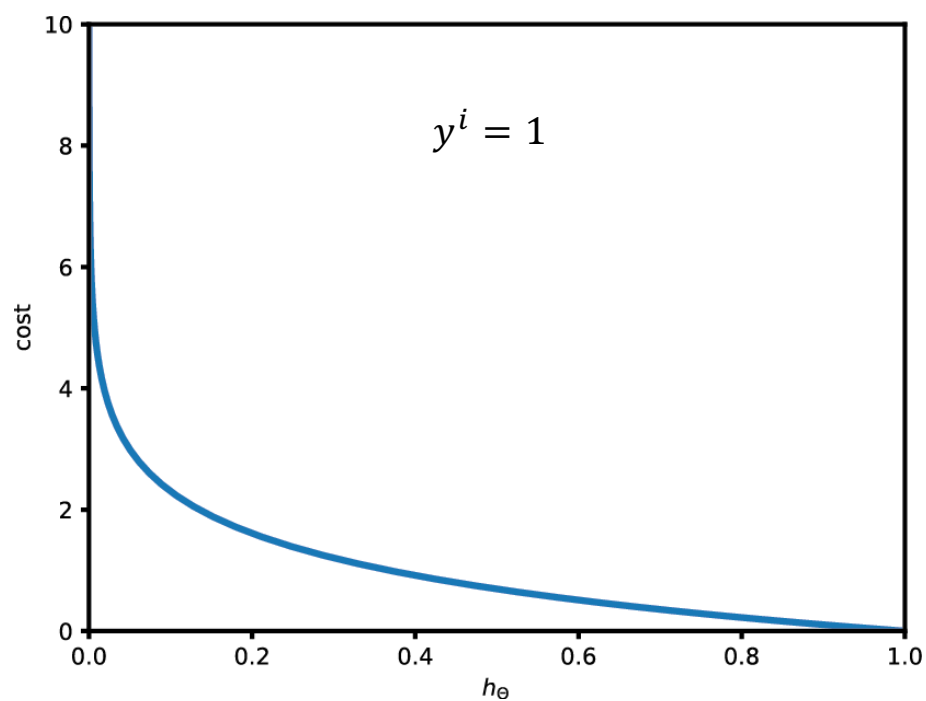
Logistic Regression – Cost Function

$$J(\theta) = \frac{1}{m} \sum_i \frac{1}{2} (h_{\theta}(x^i) - y^i)^2 = \frac{1}{m} \sum_i \text{cost}(h_{\theta}(x^i), y^i) \quad \text{cost function for linear regression}$$

This function is not convex for logistic regression $\rightarrow$ hard to find the minimum

$$\text{cost}(h_{\theta}(x^i), y^i) = \begin{cases} -\log(h_{\theta}(x^i)) & \text{if } y^i = 1 \\ -\log(1 - h_{\theta}(x^i)) & \text{if } y^i = 0 \end{cases}$$

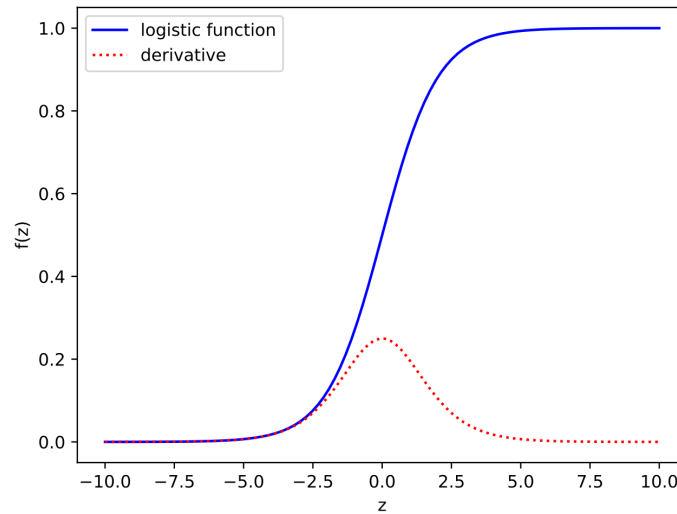
$$\text{cost}(h_{\theta}(x^i), y^i) = \begin{cases} -\log(h_{\theta}(x^i)) & \text{if } y^i = 1 \\ -\log(1 - h_{\theta}(x^i)) & \text{if } y^i = 0 \end{cases}$$



$$cost(h_{\theta}(x^i), y^i) = \begin{cases} -\log(h_{\theta}(x^i)) & \text{if } y^i = 1 \\ -\log(1 - h_{\theta}(x^i)) & \text{if } y^i = 0 \end{cases}$$

$$cost(h_{\theta}(x^i), y^i) = -y^i \log(h_{\theta}(x^i)) - (1 - y^i) \log(1 - h_{\theta}(x^i))$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^i \log(h_{\theta}(x^i)) + (1 - y^i) \log(1 - h_{\theta}(x^i))$$



$$f(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{df(z)}{dz} = \frac{d}{dz}(1 + e^{-z})^{-1} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{+1 - 1 + e^{-z}}{(1 + e^{-z})(1 + e^{-z})}$$

$$= \frac{1}{(1 + e^{-z})} \frac{1 + e^{-z} - 1}{(1 + e^{-z})} = f(z) \left(\frac{1 + e^{-z}}{1 + e^{-z}} - \frac{1}{1 + e^{-z}} \right)$$

$$= f(z)(1 - f(z))$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^i \log(h_\theta(x^i)) + (1 - y^i) \log(1 - h_\theta(x^i))$$

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m y^i \frac{1}{h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial \theta_j} - (1 - y^i) \frac{1}{1-h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial \theta_j} =$$

$$\boxed{z = \theta^T x^i}$$

$$-\frac{1}{m} \sum_{i=1}^m y^i \frac{1}{h} h(1-h) \frac{\partial z}{\partial \theta_j} - (1 - y^i) \frac{1}{1-h} h(1-h) \frac{\partial z}{\partial \theta_j}$$

$$= -\frac{1}{m} \sum_{i=1}^m y^i (1-h)x_j^i - (1 - y^i)hx_j^i = -\frac{1}{m} \sum_{i=1}^m y^i x_j^i - y^i hx_j^i - hx_j^i + y^i hx_j^i$$

$$= \frac{1}{m} \sum_{i=1}^m (h - y^i)x_j^i$$

Gradient of the cost function – Vectorial form

$$\begin{aligned}\frac{\partial J}{\partial \theta_j} &= \frac{1}{2m} \sum_i \frac{\partial (h_\theta(x^i) - y^i)^2}{\partial \theta_j} = \frac{1}{m} \sum_i (h_\theta(x^i) - y^i) \frac{\partial h_\theta(x^i)}{\partial \theta_j} \\ &= \frac{1}{m} \sum_i (h_\theta(x^i) - y^i) x_j^i\end{aligned}$$

$$\nabla_{\theta} J = \frac{1}{m} X^T (h_{\theta}(X) - Y)$$

$$\Theta' = \Theta - \frac{\alpha}{m} X^T (h_{\theta}(X) - Y) \quad \text{Gradient Descent}$$

Multi-class classification / One-Vs-All

- Train N models that classify each class versus the other classes
- Choose the class with highest probability

Multi-class classification / One-Vs-One

- Train $N*(N-1)/2$ models that classify each class versus each other class
- Choose the class that wins most duels

Softmax regression / Multinomial logistic regression

Compute a score for each class $s_k(x) = \theta_k^T x$
Each class has its own weights vector

Estimate the probability of class k using the Softmax function $p_k(x) = \frac{\exp(s_k(x))}{\sum_j \exp(s_j(x))}$

Find the parameters by minimizing the cross-entropy

$$J(\Theta) = \frac{1}{m} \sum_{i=0}^{m-1} \sum_{k=0}^{K-1} -y_k^i \log(p_k(x^i))$$

equal to 1 if sample i belongs to class k

Entropy

$$H(p) = - \sum_i p_i \log_2 p_i$$

Average amount of information that you get from one sample drawn from a given probability distribution

Tomorrow can be sunny/rainy with equal probability

The entropy is $-0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) = 1$

If I know the weather, my uncertainty decreases by 1 bit of information

The weather is classified into 4 equally likely classes.

The entropy is $-4 \cdot 0.25 \cdot \log_2(0.25) = 2$

If I know the weather my uncertainty decreases by 2 bits of information

In case that the four classes have probabilities:

- class 0 with prob. $1/2 \rightarrow -\log_2(0.5) = 1$ (is number of bits I need to encode an experiment with 2 possible outcomes)
- class 1 with prob. $1/4 \rightarrow -\log_2(0.25) = 2$ (is number of bits I need to encode an experiment with 4 possible outcomes)
- classes 2 and 3 with prob. $1/8 \rightarrow -\log_2(0.125) = 3$ (is number of bits I need to encode an experiment with 8 possible outcomes)

The entropy is the weighted average of these terms $= 0.5 + 0.25 \cdot 2 + 0.125 \cdot 3 + 0.125 \cdot 3 = 1.75$ bits

Cross-entropy

predicted distribution

$$H(p, q) = - \sum_i p_i \log_2 q_i = - \sum_i p_i \log_2 \frac{p_i}{p_i} q_i$$

true distribution

$$H(p, q) = - \sum_i p_i \log_2 p_i + \sum_i p_i \log_2 \frac{p_i}{q_i} = H(p) + KL(p, q)$$

how many bits I need
to encode the
outcome using the
wrong model q

entropy of the true
distribution

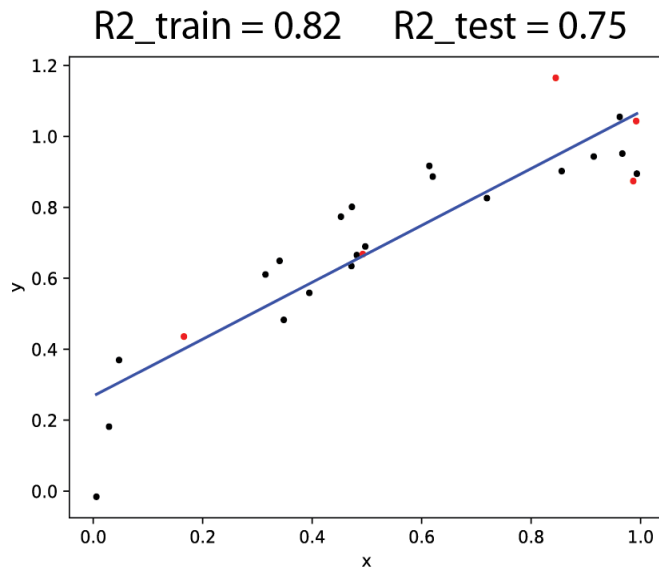
KL divergence:
mismatch if p is
approximated by q

So minimizing the cross-entropy corresponds to
minimize the KL divergence

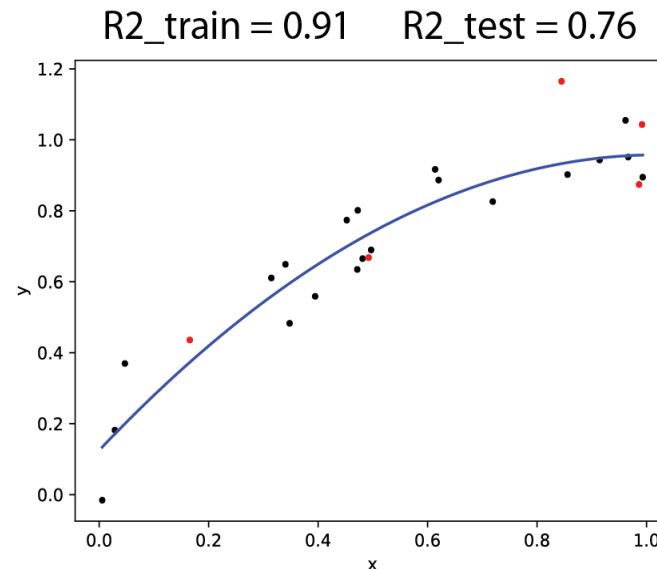
underfitting
aka: high bias



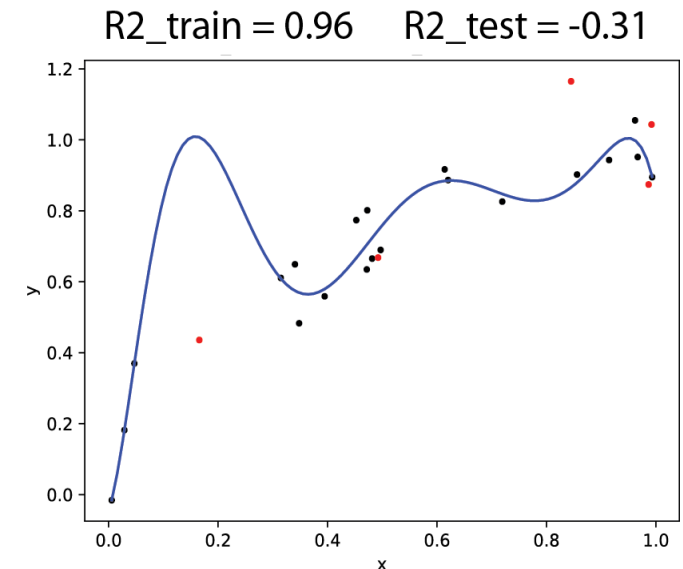
overfitting
aka: high variance



$$h_{\theta} = \theta_0 + \theta_1 x$$



$$h_{\theta} = \theta_0 + \theta_1 x + \theta_2 x^2$$



$$h_{\theta} = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_{10} x^{10}$$

How to avoid overfitting

- Manually reduce the number of features (features' selection)
- Regularization = keep all the features but reduce the value of the parameters

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=0}^{m-1} (h_{\theta}(x^i) - y^i)^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

reduce the distance from the true values in the training set

regularization parameter

keep the model simple

The first parameter (intercept) is not usually included in this sum

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=0}^{m-1} (h_{\theta}(x^i) - y^i)^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_i (h_{\theta}(x^i) - y^i) x_j^i \quad j = 0$$

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_i (h_{\theta}(x^i) - y^i) x_j^i + \frac{\lambda \theta_j}{m} \quad j = 1, \dots, n$$

$$\theta_j = \theta_j - \alpha \left. \frac{\partial J}{\partial \theta_j} \right|_{\theta} = \theta_j \left(1 - \alpha \frac{\lambda}{m} \right) - \frac{\alpha}{m} \sum_{i=0}^{m-1} (h_{\theta}(x^i) - y^i) x^i$$

“shrink” the previous estimate of the parameter

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_i (h_{\theta}(x^i) - y^i) x_j^i \quad j = 0$$

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_i (h_{\theta}(x^i) - y^i) x_j^i + \frac{\lambda \theta_j}{m} \quad j = 1, \dots, n$$

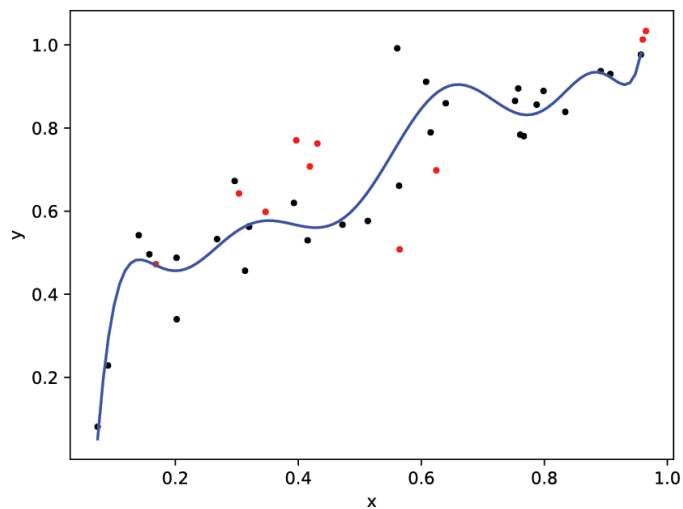
$$\nabla_{\theta} J = \frac{1}{m} [X^T (X\Theta - Y) + \lambda I_o \Theta] \quad I_o = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\Theta = (X^T X + \lambda I_o)^{-1} X^T Y$$

$$h_{\theta} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_{10} x^{10}$$

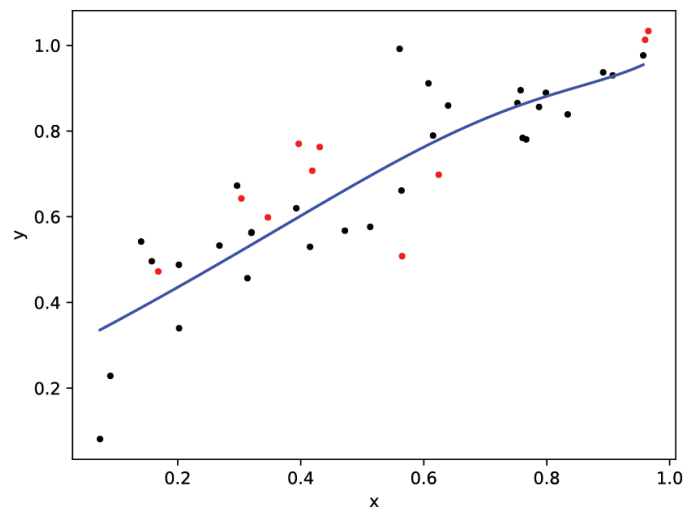
$\lambda = 0$

R2_train = 0.90 R2_test = 0.31



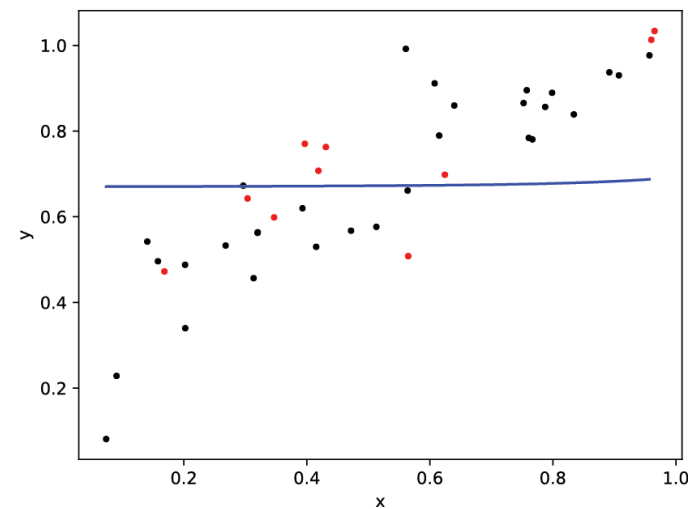
$\lambda = 1.0$

R2_train = 0.80 R2_test = 0.55



$\lambda = 10000$

R2_train = 0.02 R2_test = -0.01



Regularization methods

- Ridge (L2 regularization)

$$\frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

- Lasso (L1 regularization)

$$\frac{\lambda}{2m} \sum_{j=1}^n |\theta_j|$$

weights of non-relevant features
are forced to be exactly zero

- Elastic nets
(L1 + L2 regularization)

$$\frac{\lambda}{2m} \left((1-r) \sum_{j=1}^n \theta_j^2 + r \sum_{j=1}^n |\theta_j| \right)$$

$r \in [0,1]$

Firth's Logistic Regression

Developed to deal with the problems that arise when using Logistic Regression for

- small datasets
- imbalanced datasets (the loss-function is dominated by the major class)
- complete separation of classes along one feature (the model tends to $\pm\infty$ depending on values of the independent variables, and consequently the parameters diverge)

$$J(\theta) = J'(\theta) - \frac{1}{2} \log |I(\theta)|$$


part of the cost function related only to the distance from the data

determinant of the Fisher's information matrix

Fisher Information Matrix

It measures how sensitive is the likelihood to changes in parameters

The idea is that if a small change in a parameter greatly change the likelihood, that parameter is highly informative

log-likelihood (or likelihood)


$$I(\theta) = -\mathbb{E} \left[\frac{\partial^2 l}{\partial \theta \partial \theta^T} \right]$$

$$I(\theta)_{ij} = -\mathbb{E} \left[\frac{\partial^2 l}{\partial \theta_i \partial \theta_j} \right]$$

Suppose the output is sampled from a Bernoulli distribution with probability p_i depending on the input features according to the logistic function

$$y_i \sim \text{Bernoulli}(p_i) \quad p_i(x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$y_i \sim p_i^{y_i} (1 - p_i)^{1-y_i}$$

since samples are independent, the likelihood can be calculated as:

$$L(\theta) = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i}$$

log-likelihood

$$l(\theta) = \sum_i y_i \log p_i + (1 - y_i) \log(1 - p_i)$$

$$\frac{\partial l}{\partial \theta_j} = \sum_i (y_i - p_i) x_{ij}$$

$$\nabla_{\theta} l = X^T (Y - P)$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^i \log(h_{\theta}(x^i)) + (1 - y^i) \log(1 - h_{\theta}(x^i))$$

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h - y^i) x_j^i$$

$$X = \begin{pmatrix} x_0^0 & \dots & x_n^0 \\ \vdots & \ddots & \vdots \\ x_0^{m-1} & \dots & x_n^{m-1} \end{pmatrix}$$

$$Y = (y_1, \dots, y_m)^T$$

$$P = (p_1, \dots, p_m)^T$$

$$\frac{\partial^2 l}{\partial \theta_k \partial \theta_j} = \frac{\partial}{\partial \theta_k} \left(\sum_i (y_i - p_i) x_{ij} \right) = - \frac{\partial}{\partial \theta_k} \left(\sum_i x_{ij} p_i \right)$$

$$\frac{\partial^2 l}{\partial \theta_k \partial \theta_j} = - \sum_i x_{ij} \frac{\partial p_i}{\partial z} \frac{\partial z}{\partial \theta_k} = - \sum_i x_{ij} x_{ik} p_i (1 - p_i)$$

$$H(\theta) = -X^T W X$$

$$I(\theta) = X^T W X$$

$$W = \begin{pmatrix} p_1(1-p_1) & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & p_m(1-p_m) \end{pmatrix}$$

$$p_i(x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$f(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{df(z)}{dz} = f(z)(1 - f(z))$$

For a model with a single parameter, in the proximity of the maximum:

$$l(\theta + \delta) \approx l(\theta) + \cancel{\frac{\partial l}{\partial \theta} \delta} + \frac{1}{2} \frac{\partial^2 l}{\partial \theta^2} \delta$$

zero in the maximum of the likelihood

it's minus the information matrix

If the second derivative is high

→ a small change in the parameter causes a high change in the log-likelihood

→ high sensibility to the parameter

For a model with a n parameters:

$$l(\theta + \delta) \approx l(\theta) + \cancel{\nabla_{\theta} l \delta} + \frac{1}{2} \delta^T H_{\theta} \delta$$

$$l(\theta + \delta) \approx l(\theta) - \frac{1}{2} \delta^T I(\theta) \delta$$

In a similar way here, if the determinant of the information matrix is high, there's high curvature of the log-likelihood in the proximity of the maximum, which corresponds to high sensibility to the values of the parameters. Vice versa low-curvature means that there's low-dependency on the parameter values.

$$J(\theta) = J'(\theta) - \frac{1}{2} \log |I(\theta)|$$

So, the last term of the cost-function penalizes regions where the likelihood is flat, and consequently regions where the parameters are poorly defined.